Software

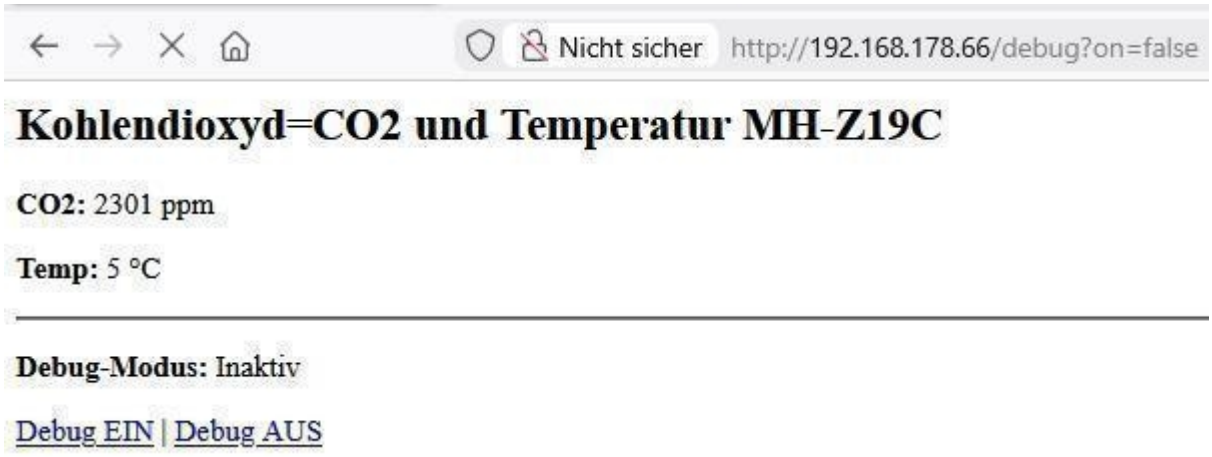
The software , Versions 1 to 4 , is available as a complete zip file.

The programs were (currently) developed entirely in the **Python programming language** . Well, what more can I say? AI at work. Additional information can be found in the file Hardware-info.pdf. To prevent any interference issues, I distributed the sensors and their associated software across four Raspberry Pi Pico 2W units . There are three program variations for each sensor available: one for reading the sensor data only, one with display on the OLED SSD1306 display, and one with data transmission via Wi-Fi.

The software is being expanded step by step from Version 1 to Version 4.

Software Version 1

Folder: Version-1 . This version is ideal for beginners and for testing purposes. The data from each sensor is displayed directly on the mini-OLED display. With a Wi-Fi connection, each Raspberry Pi Pico 2W essentially acts as a "mini web server."
The RadiationD-V1.1 (CAJOE) Geiger counter is not Wi-Fi enabled in Version-1 .
However, this software Version-1 has many disadvantages and is not suitable for continuous operation because: Multiple Picos acting as "mini web servers" on the Wi-Fi network → unstable during continuous operation. Each Pico = its own web server; the browser accesses each Pico directly. Each Pico has Wi-Fi + socket + HTML
– -> **connection drops and timeouts.** For further information, here is an example of how the data appears in the browser:



Software Version 2

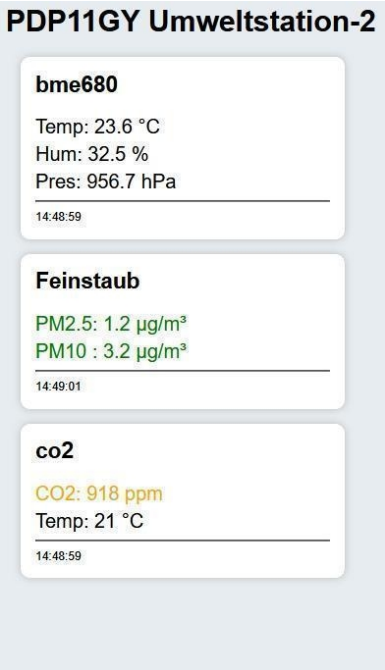
Folder: Version-2 client/server architecture is required for stable, continuous operation . An additional server system is now necessary. I opted for the **Raspberry Pi Zero 2W** system for the weather station. Here are the individual installation steps:

- The operating system can be easily downloaded using the Raspberry Pi Imager and installed on the microSD card. I chose Raspberry Pi OS Lite (64-bit). Please remember to enable SSH support. - I temporarily connected a monitor and created a user (pi) with a password. In my case, it was pi, and I had to permanently enable SSH with: `sudo systemctl enable ssh` and `sudo systemctl start ssh`. Query the IP address with `ifconfig -a`. From now on, my server was accessible via SSH (even from Windows), e.g., ssh pi@192.168.178.72 .

- To avoid SSH hangs: edit the `/etc/ssh/sshd_config` file with `sudo nano /etc/ssh/sshd_config`. At the very bottom, add the line ` **IPQoS cs0 cs0**` and then restart the SSH service with: `sudo systemctl restart ssh`.

- Install Python 3 and Flask software with: `sudo apt update` and `sudo apt install python3-pip -y pip3 install flask`. If there's an error: `sudo apt install python3-flask -y`.

All client/server software is located in the **`Version-2`** **folder.** Copy the weather station server software, **`server.py`, to the server system.** The weather station can now be started: with **python3 server.py.** Next, copy the software for the sensors to the corresponding PICOs: Beforehand, the Wi-Fi ID and password must be set in the Python program, as well as the IP address of the server system, e.g., for the CO2 sensor: Weather_V2/MH-Z19C-Client_V1.py : SERVER = " http://192.168.178.72:5000 /data". Now, copy (save as) the program, named main.py to the target PICO. Once all clients are started, the data can be viewed in a browser at the address **http://192.168.178.72:5000** _as follows:



The limit values for CO2 and particulate matter are represented by **green** / **orange** / **red** .

For info: I won't go into further detail about adjustments/optimizations in the Raspberry Pi operating system here, as this is already extensively documented elsewhere. The problems with the simultaneous startup of the PICO devices, especially the CO2 sensor, are **guaranteed to be fixed** . Issues such as "system hangs" occurred specifically when connecting to the Wi-Fi network and were resolved by using appropriate watchdog timers. All status messages are displayed on the OLED display at startup.

Software Version 3

Folder: Version-3 weather station accessible outside of WLAN, SSD disk for data storage and integration of the Geiger counter module.
Implemented using the open-source reverse proxy service **ngrok** .
Installation is simple:

wget https://bin.equinox.io/c/bNyj1mQVY4c/ngrok-v3-stable-linux-arm.zip
unzip ngrok-v3-stable-linux-arm.zip
sudo mv ngrok /usr/local/bin
Next, open your browser and go to: https://dashboard.ngrok.com/signup.
You will receive your token here: https://dashboard.ngrok.com/get-started/your-authtoken
Activate the token with: `ngrok config add-authtoken TOKEN`.
Start the token with: `ngrok http 5000` . You will then receive the following message:
Forwarding: **https://stony-outlying-unaltered.ngrok-free.dev -> http://localhost:5000**.
You can also control access to the data with a password. I have included the server program ` **server(_V3).py**` as an example in the `/Version-3` folder. For password protection, the line `@requires_auth` at the end of the server program is crucial . I implemented auto-start using systemctl. My stations are now accessible from outside the network.

External SSD Disk

The Raspberry Pi Zero 2W exhibited strange behavior when implemented with an external SSD. Ultimately, it depends on the quality of the power supply. For me, adding the entry ` **program_usb_boot_timeout=2`** to the file ` **/boot/firmware/config.txt`** resolved the issue.
The actual setup/installation of an external drive is described in detail elsewhere. In my case, I created the folders `/mnt/ssd/data`, `/mnt/ssd/logs`, and `/mnt/ssd/cam`. Don't forget the following command: `sudo chown -R pi:pi /mnt/ssd .`
To enable data logging, the following must be added to the `server.py` program (it's already included in the example `server.py` program):
`DATA_PATH = "/mnt/ssd/data/"` and `DATA_FILE = DATA_PATH + "data.json"`

Geiger counter RadiationD-V1.1 (CAJOE)

This sensor module now also has WLAN connectivity, together with a BMP280 sensor to determine a possible correlation with air pressure/humidity.

Software Version 4

Folder: Version-4 : **Full configuration** with **both** climate stations, Station-1 and Station-2. The primary **SERVER1** is located in **Station-1** . In my case, it's equipped with a 520GB SSD disk, accessible from outside via ngrok. It receives and stores all data from the sensors, including those in Station-2. The respective sensor programs, along with the server program version 4 (server_V4.py), are located in the Station-1 subfolder. Server-2 **in** Station **-2** , in my case, uses server version V2 (server_V2.py) and contains customized sensor programs in version V4, also located in the Station-2 subfolder. Each station uses three Raspberry Pi 2W microcontrollers. Station-1 always stores all data on SERVER1. SERVER2 in Station-2 checks if SERVER1 in Station-1 is online and then sends the sensor data to Station-1, while also storing the data on Station-2, which is equipped with a 128GB SSD disk. The Geiger counter always attempts to send data to both stations.
This concept is very flexible and can be expanded and extended at any time by adding another station or additional/different sensors, each with a Raspberry Pi Pico 2W.
Example in folder: Extensions/UV+GAS

Version 4_2

Setting the storage location(=SSD) and path:

SSD-path: /mnt / ssd / data /
Partitionierung(example: 15-JUN-2026): 2026_KW24 / 2026-06-15 / daten.json
Server versionen: server_V4_2.py und server_V2_2.py

Version 4_3

Implementation of the radiation station, additionally with UV sensor.

Server versions: server_V4_3.py and server_V2_3.py

Version 4_4

Camera support, implementation:

A " Camera " button is implemented in server/Station-1. and Station-2. This button opens a new tab in the browser and displays an image taken with the camera on the Raspberry Pi Zero W. This image is updated every 5 minutes, and one image is saved daily at 2 PM to: /mnt/ssd/cam/ (e.g., 2026-07-13_1400.jpg).

The rpicam software must be installed for this to work: `**sudo apt install rpicam-apps**`

Furthermore, password prompts should be disabled. A key must be generated for this, e.g., on Windows: `**ssh-keygen -t ed25519**` (press return several times). The key will then be located in **C:\Users\<username>\.ssh** . Add this key: ` **mkdir -p ~/.ssh , nano**

~/.ssh/authorized_keys .`

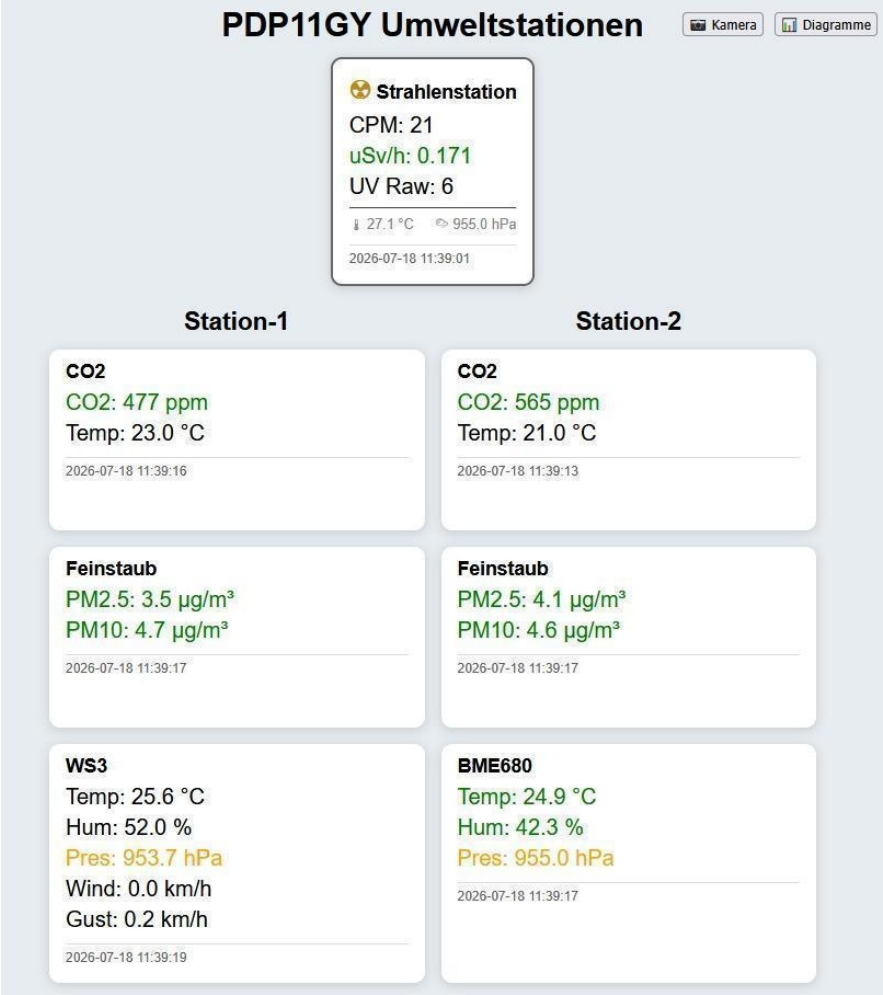
Finally, change the access rights: **chmod 700 ~/.ssh , chmod 600 ~/.ssh/authorized_keys**

and, two **more entries are needed in the crontab , created with crontab -e :**

*/5 * * * * /usr/bin/rpicam-still --nopreview -o /mnt/ssd/cam/bild.jpg >/dev/null 2>&1
0 14 * * * /bin/cp /mnt/ssd/cam/bild.jpg /mnt/ssd/cam/$(/bin/date +\%Y-\%m-\%d_\%H\%M).jpg

Server Versionen: server_V4_4.py und server_V2_4.py

The following image shows the output of version 4_5 with remote access via ngrok, also possible on mobile phones from anywhere in the world.



More details: **Climatestation.html**

Please send your suggestions and comments. I would be very happy to hear from you.

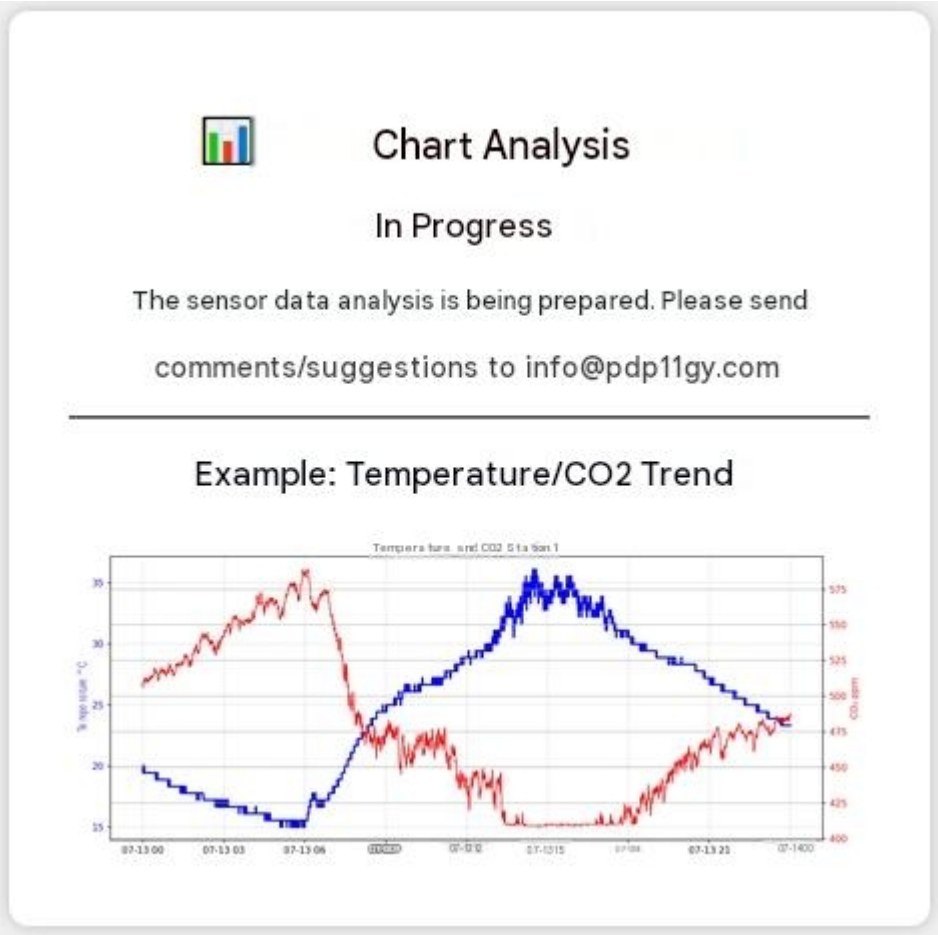
Please email **info@pdp11gy.com**

Further software versions

planned/in progress:

Graphical analysis , e.g., with Python libraries like matplotlib and plotly. There are **many** options for graphical analysis, including ready-made programs and complete tools. I take a rather neutral view of the topic but would be very happy to receive suggestions/recommendations.
However, for now, I have opted for the open-source product **matplotlib** .
An example is available in the " **Diagrams** " folder; see the file **Diagram-Example.pdf.**
To be able to read/copy data , I use a Windows **network drive . For this, Samba must** be installed on the Raspberry Pi server using **`sudo apt install samba` and** configured in the file `/etc/samba/smb.conf` **.** In my case, this allows me to analyze and display the data much faster. A detailed graphical analysis can only be performed after a longer measurement period.
Prepared with server versions: **server_V4_5.py** and **server_V2_5.py**

Clicking the diagram button displays the following image: :



My idea is to create a calendar icon and an icon for each sensor as well. This would allow a user to select a date and choose two to four sensors, then visualize the interdependencies graphically.

Unfortunately, I haven't received any messages so far(Jun-2026) for expansion/suggestions and **collaboration** … Too bad.... but I'm working on it